HPC5

Implement HPC application for AI/ML domain.

```
#include <iostream>      // for input/output
#include <vector>        // for dynamic arrays
#include <cmath>         // for sqrt()
#include <algorithm>     // for sort()
#include <omp.h>         // for OpenMP (parallelism)

using namespace std;

// 🔹 Structure to represent a data point
struct Point {
    double x, y;   // coordinates
    int label;     // class label (0 or 1)
};

// 🔹 Function to calculate Euclidean distance
double euclideanDistance(const Point &a, const Point &b) {
    // distance = √((x1-x2)^2 + (y1-y2)^2)
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}

// 🔹 KNN classification function
int knn(const vector<Point> &dataset, const Point &query, int K) {

    int n = dataset.size();   // total number of data points

    // vector to store (distance, label)
```

```
    vector<pair<double, int>> dist(n);

    // 🔵 PARALLEL DISTANCE CALCULATION
    // Each thread computes distance independently
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        dist[i] = {euclideanDistance(dataset[i], query), dataset[i].label};
    }

    // 🔹 Sort distances in ascending order
    sort(dist.begin(), dist.end());

    // 🔹 Count labels among K nearest neighbors
    int count0 = 0, count1 = 0;

    // 🔵 PARALLEL VOTING USING REDUCTION
    #pragma omp parallel for reduction(+:count0, count1)
    for (int i = 0; i < K; i++) {
        if (dist[i].second == 0)
            count0++;   // class 0
        else
            count1++;   // class 1
    }

    // 🔹 Return class with majority votes
    return (count1 > count0) ? 1 : 0;
}
```

```
int main() {

    // 🔹 Dataset (training data)
    vector<Point> dataset = {
        {1,2,0}, {2,3,0}, {3,1,0},   // Class 0
        {6,5,1}, {7,7,1}, {8,6,1}    // Class 1
    };

    // 🔹 Query point (unknown class)
    Point query = {5, 5, -1};

    int K = 3;   // number of nearest neighbors

    // 🔹 Call KNN function
    int result = knn(dataset, query, K);

    // 🔹 Output result
    cout << "Query Point: (" << query.x << ", " << query.y << ")\n";
    cout << "Predicted Class: " << result << endl;

    return 0;
}
```

✅ 🔹 HEADER FILES

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <omp.h>
```

👉 Explain like this:

- iostream → for input/output (cout)
- vector → to store dataset
- cmath → for sqrt() (distance calculation)
- algorithm → for sort()
- omp.h → for **parallel programming (OpenMP)**

✅ 🔹 USING NAMESPACE

```
using namespace std;
```

👉 Avoid writing std:: again and again

✅ 🔹 STRUCTURE

```
struct Point {
    double x, y;
    int label;
};
```

👉 Explain:

"This structure stores each data point with x, y coordinates and its class label."

✅ 🔹 DISTANCE FUNCTION

```
double euclideanDistance(const Point &a, const Point &b) {
    return sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
}
```

👉 Explain:

"This function calculates Euclidean distance between two points."

👉 Formula:

√((x1-x2)^2 + (y1-y2)^2)

✅ 🔹 **KNN FUNCTION**

int knn(const vector<Point> &dataset, const Point &query, int K)

👉 Explain:

"This function performs KNN classification."

🔹 **Create Distance Array**

vector<pair<double, int>> dist(n);

👉 Explain:

"This stores distance and corresponding class label."

🔹 **PARALLEL DISTANCE CALCULATION**

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    dist[i] = {euclideanDistance(dataset[i], query), dataset[i].label};
}
```

👉 Explain:

"This loop calculates distance of query point from all dataset points in parallel using OpenMP."

🔹 **SORT DISTANCES**

sort(dist.begin(), dist.end());

👉 Explain:

"This sorts distances in ascending order so nearest points come first."

🔹 **COUNT NEIGHBORS**

int count0 = 0, count1 = 0;

👉 Counters for classes

🔹 **PARALLEL VOTING**

```
#pragma omp parallel for reduction(+:count0, count1)
for (int i = 0; i < K; i++) {
    if (dist[i].second == 0)
        count0++;
    else
        count1++;
}
```

👉 Explain:

"This counts class labels of K nearest neighbors using parallel reduction."

🔹 **RETURN RESULT**

```
return (count1 > count0) ? 1 : 0;
```

👉 Explain:

"Returns the class with maximum votes."

✅ 🔹 **MAIN FUNCTION**

```
vector<Point> dataset = {
    {1,2,0}, {2,3,0}, {3,1,0},
    {6,5,1}, {7,7,1}, {8,6,1}
};
```

👉 Explain:

"This is the training dataset with two classes."

```
Point query = {5, 5, -1};
```

👉 Explain:

"This is the test point whose class we want to predict."

```
int K = 3;
```

👉 Number of neighbors

---

int result = knn(dataset, query, K);

👉 Call classification function

---

cout << "Predicted Class: " << result << endl;

👉 Print output

🎯 FINAL VIVA EXPLANATION (BEST)

“This program implements K-Nearest Neighbors classification. It calculates distances between query and dataset points, sorts them, selects K nearest neighbors, and performs majority voting. OpenMP is used to parallelize distance computation and voting, improving performance.”

🔥 SUPER SHORT VERSION (IF TIME IS LESS)

“KNN classifies a point based on nearest neighbors. Distance calculation and voting are parallelized using OpenMP.”

💡 EXTRA SMART LINE

“This is an example of HPC in AI/ML where computation is parallelized to improve performance.”

✅ INPUT (from your code)

```
dataset = {
 {1,2,0}, {2,3,0}, {3,1,0},
 {6,5,1}, {7,7,1}, {8,6,1}
};

query = {5,5}
K = 3
```

🔁 STEP 1: DISTANCE CALCULATION (Parallel Loop)

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
   dist[i] = {distance(dataset[i], query), label};
}
```

👉 Each thread calculates distance independently

🔢 Computation Table

i	Point	Distance	Label
0	(1,2)	√(16+9)=5	0
1	(2,3)	√(9+4)=3.6	0
2	(3,1)	√(4+16)=4.47	0
3	(6,5)	√(1+0)=1	1
4	(7,7)	√(4+4)=2.82	1
5	(8,6)	√(9+1)=3.16	1

👉 Stored as:

dist = [(5,0), (3.6,0), (4.47,0), (1,1), (2.82,1), (3.16,1)]

🔁 STEP 2: SORT DISTANCES

```
sort(dist.begin(), dist.end());
```

👉 Sorted result:

[(1,1), (2.82,1), (3.16,1), (3.6,0), (4.47,0), (5,0)]

🔁 STEP 3: TAKE K NEAREST

K = 3

👉 First 3 elements:

(1,1), (2.82,1), (3.16,1)

🔁 STEP 4: VOTING (Parallel Reduction)

```
#pragma omp parallel for reduction(+:count0, count1)
for (int i = 0; i < K; i++)
```

👉 Count labels:

Neighbor	Label
1st	1
2nd	1
3rd	1

👉 Result:

count1 = 3
count0 = 0

✅ STEP 5: FINAL RESULT

```
return (count1 > count0) ? 1 : 0;
```

👉 Output:

Predicted Class = 1

⚙️ PARALLEL EXECUTION VIEW

🔹 **Distance Loop**

Thread	Work
T1	dist[0]
T2	dist[1]
T3	dist[2]
...	...

🔹 **Voting Loop**

Thread	Work
T1	check neighbor 1
T2	check neighbor 2
T3	check neighbor 3

🎯 HOW TO EXPLAIN IN VIVA

"First, distances from query point to all dataset points are computed in parallel. Then distances are sorted, and K nearest neighbors are selected. Finally, majority voting is performed to predict the class."

▶️ ✅ COMMANDS TO RUN YOUR KNN CODE (OPENMP)

🔹 Step 1: Go to your file folder

(Example if file is on Desktop)

```
cd ~/Desktop
```

🔹 Step 2: Compile (IMPORTANT)

```
g++ -fopenmp knn.cpp -o knn
```

👉 -fopenmp enables parallel execution

🔹 Step 3: Run

```
./knn
```

✅ 🎯 EXPECTED OUTPUT

Query Point: (5, 5)
Predicted Class: 1

🧠 WHY OUTPUT IS 1

👉 Nearest 3 points:

- (6,5) → class 1
- (7,7) → class 1
- (8,6) → class 1

👉 Majority = **Class 1**

⚠️ IF ERROR COMES

❌ Error: omp.h not found

Run:

```
sudo apt install libomp-dev
```

❌ Forgot -fopenmp

👉 Then parallel won't work

❌ File not found

Check file name:

```
ls
```

🔥 BONUS (CHECK PARALLEL WORKING)

```
export OMP_NUM_THREADS=4
./knn
```

👉 Runs using 4 threads

🎯 WHAT TO SAY IN VIVA AFTER RUN

"The program successfully classifies the query point using KNN. Parallelism is applied in distance computation and voting using OpenMP."

💡 QUICK SUMMARY

Step	Command
Compile	g++ -fopenmp knn.cpp -o knn
Run	./knn

✅ 📁 FILENAME

You can name your file anything, but **best recommended name is:**

knn.cpp

▶️ COMPLETE FLOW

🔹 Step 1: Create file

nano knn.cpp

👉 Paste your code
👉 Save: Ctrl + X → Y → Enter

🔹 Step 2: Compile

g++ -fopenmp knn.cpp -o knn

🔹 Step 3: Run

./knn

💡 IMPORTANT

👉 If you name file differently, use same name:

Example:

g++ -fopenmp mycode.cpp -o mycode
./mycode

✅ 📁 FILENAME

You can name your file anything, but **best recommended name is:**

knn.cpp

▶️ COMPLETE FLOW

🔹 Step 1: Create file

nano knn.cpp

👉 Paste your code
👉 Save: Ctrl + X → Y → Enter

🔹 Step 2: Compile

g++ -fopenmp knn.cpp -o knn

🔹 Step 3: Run

./knn

💡 IMPORTANT

👉 If you name file differently, use same name:

Example:

g++ -fopenmp mycode.cpp -o mycode
./mycode